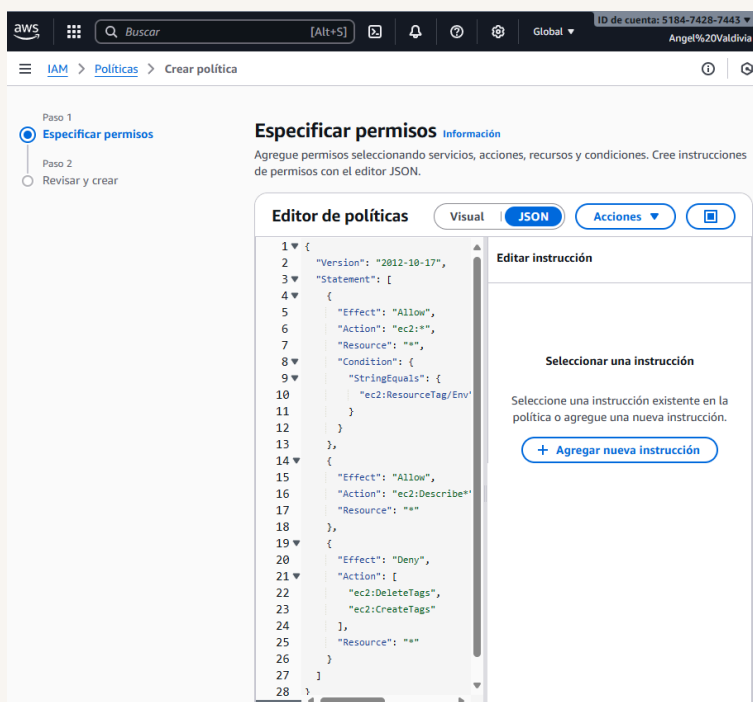




Cloud Security with AWS IAM



Angel G. Valdivia H.





Introducing Today's Project!

In this project, I'll demonstrate how AWS IAM manages access to EC2 instances using user groups and policies. I aim to learn secure cloud permissions, enforce least privilege, and control access across development and production environments.

Tools and concepts

Services I used were EC2 and IAM. Key concepts I learnt include tagging for environment control, least privilege access, JSON policy creation, IAM user and group setup, and secure onboarding using account aliases and scoped permissions.

Project reflection

This project took me approximately 1 hour. The most challenging part was designing the IAM policy with precise tag-based conditions. It was most rewarding to see the intern's access work exactly as intended—secure, scoped, and functional.



Tags

Tags are keywords that organize and classify content. They improve search, group related items, and simplify navigation. In cloud projects, tags label resources like EC2 for cost tracking, access control, and environment identification

The tags I used on my two EC2 instances are "development" and "production".

The screenshot shows the AWS Management Console interface for launching an EC2 instance. The page title is 'Lanzar una instancia'. A blue banner at the top contains a message about first-time users and a 'Realice un recorrido' button. Below this, the 'Nombre y etiquetas' section is expanded, showing a table with two rows of tags. The first row has a key 'Name' and a value 'nextwork-dev-Ang'. The second row has a key 'Env' and a value 'development'. Each row has an 'Eliminar' button. At the bottom of the section is an 'Agregar nueva etiqueta' button and a note that up to 48 tags can be added. The right side of the console shows the 'Tipos de recurso' section with a dropdown menu and an 'Instancias' button.

Clave	Valor	Tipos de recurso
Name	nextwork-dev-Ang	Seleccionar tipos de r... Instancias
Env	development	Seleccionar tipos de r... Instancias



IAM Policies

IAM policies are JSON-based rules that define permissions for AWS resources. They control who can access what, under which conditions—ensuring secure, role-based access to services like EC2, S3, and more across cloud environments.

The policy I set up

For this project, I've set up a policy using JSON. It gave me full control over the permissions structure, allowing precise access to the development EC2 instance for the intern while keeping production resources secure.

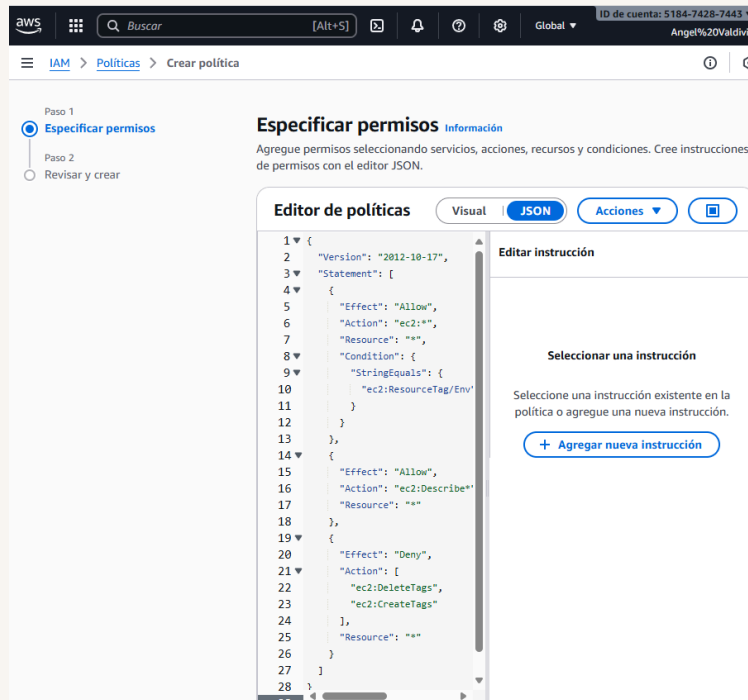
I've created a policy that allows EC2 actions only on resources tagged as "development," permits read-only access to all EC2 resources, and blocks tag creation or deletion—ensuring secure, limited access for the intern.

When creating a JSON policy, you have to define its Effect, Action and Resource.

The Effect defines whether access is allowed or denied. Action specifies the operations (like `ec2:StartInstances`). Resource identifies the AWS items the policy applies to, such as specific EC2 instances or all resources.



My JSON Policy

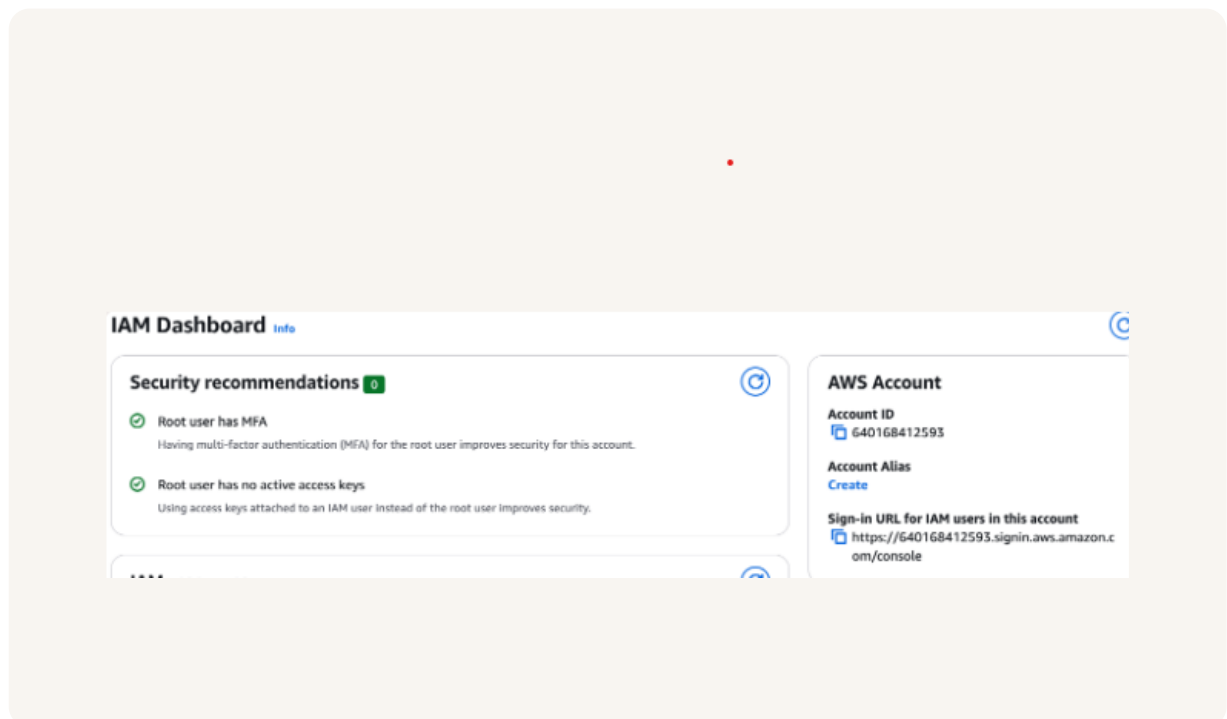




Account Alias

An account alias is a custom name for your AWS account that replaces the default numeric ID. It simplifies login by letting users access the console using a recognizable URL like <https://alias.signin.aws.amazon.com/console>.

Creating an account alias took me less than 5 minutes. Now, my new AWS console sign-in URL is easier to remember and share





IAM Users and User Groups

Users

IAM users are individual identities in AWS with credentials and permissions. Each user can log in, access resources, and perform actions based on assigned policies—ideal for team members like interns or developers needing secure access.

User Groups

IAM user groups are collections of IAM users that share the same permissions. They simplify access management by letting you assign policies to the group instead of individual users—ideal for teams like interns or developers.

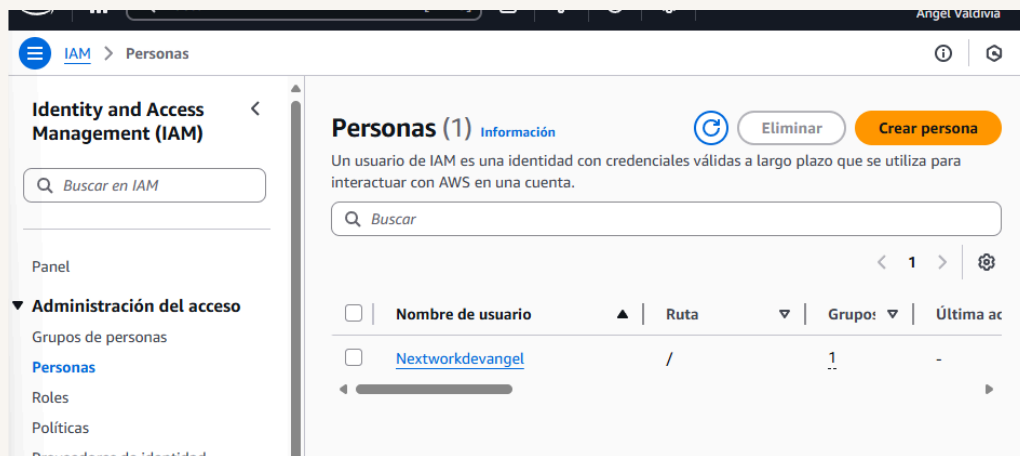
I attached the policy I created to this user group, which means all users in the group inherit the same permissions—granting access to development EC2 resources while restricting tag changes and protecting production environments.



Logging in as an IAM User

The first way is to email the user their login credentials. The second way is to download and securely share the .csv file with their access key and password. Both methods ensure the user can access the AWS console safely.

Once I logged in as my IAM user, I noticed an “Access Denied” error for service catalog applications. This was because the policy only allows EC2 access for development, not broader services—confirming least privilege is enforced.

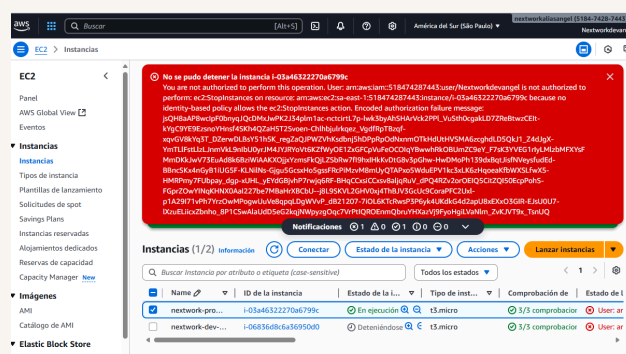




I tested my JSON IAM policy by attempting to stop both EC2 instances. The intern user was able to interact with the development instance but was denied access to the production one, confirming the policy works as intended.

Stopping the production instance

When I tried to stop the production instance, access was denied. This was because the IAM policy restricts EC2 actions to resources tagged as “development,” preventing interns from modifying production resources.

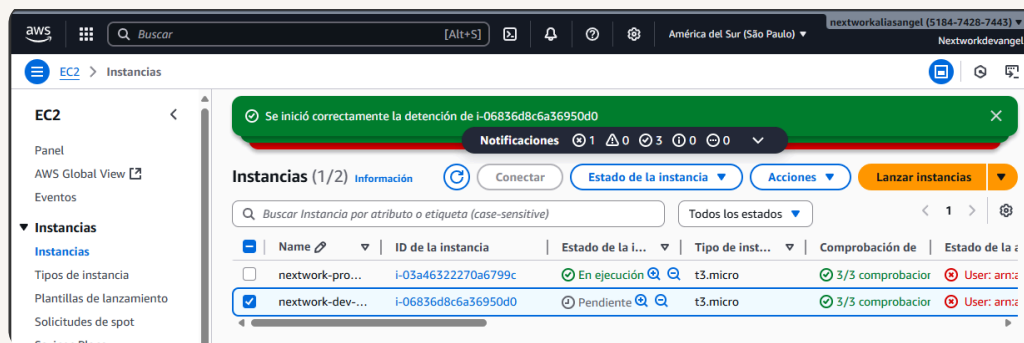




Testing IAM Policies

Stopping the development instance

Next, when I tried to stop the development instance, the action succeeded. This was because the IAM policy allows EC2 operations on resources tagged with “Env: development,” confirming correct access for the intern.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

